# RAG Test Document — Embedding Test

This document is designed to test PDF text extraction, chunking, OpenAI text embeddings, PostgreSQL vector storage, and semantic search in a Retrieval-Augmented Generation (RAG) application.

## RAG Embedding Test — Page 1

## Section: Retrieval-Augmented Generation

Retrieval-Augmented Generation combines information retrieval with a large language model. The application first retrieves relevant information from documents and then provides that information to the language model as context.

## Section: Text Chunking

Long documents are divided into smaller chunks before embedding. Chunking makes semantic search more precise because each vector represents a manageable section of text. Overlapping chunks can preserve context across boundaries.

## Section: Embeddings

An embedding is a numerical representation of text meaning. An embedding model converts a sentence, paragraph, or document chunk into a vector containing floating-point values. Texts with similar meanings tend to have nearby vectors.

## Section: PostgreSQL and pgvector

A RAG application can store document chunks and their embeddings in PostgreSQL. The pgvector extension adds vector data types and similarity-search capabilities. The application can compare a query embedding with stored document embeddings.

## Section: Semantic Search

Semantic search focuses on meaning rather than exact keyword matches. A question about finding relevant information can retrieve a chunk about vector similarity even when the exact words from the question are not present.

## Section: Clean Architecture

Clean Architecture separates business rules from infrastructure concerns. The domain layer remains independent of databases and external services. Application use cases define behavior, while infrastructure provides implementations.

## Section: CQRS

Command Query Responsibility Segregation separates state-changing commands from read-only queries. MediatR can dispatch commands and queries to their handlers.

## Section: Document Processing Pipeline

A typical RAG pipeline starts with a PDF or text upload, followed by metadata storage, text extraction, chunking, embedding generation, vector storage, and semantic retrieval.

## Section: Similarity

Vector similarity measures how close two embedding vectors are. Cosine similarity is commonly used for text embeddings. Higher similarity generally indicates that two pieces of text are more semantically related.

Test content 1: embeddings, vectors, chunks, semantic search, PostgreSQL, pgvector, retrieval, context, and language models are repeated intentionally so that chunking and semantic-search behavior can be tested.

# RAG Embedding Test — Page 2

## Section: Retrieval-Augmented Generation

Retrieval-Augmented Generation combines information retrieval with a large language model. The application first retrieves relevant information from documents and then provides that information to the language model as context.

## Section: Text Chunking

Long documents are divided into smaller chunks before embedding. Chunking makes semantic search more precise because each vector represents a manageable section of text. Overlapping chunks can preserve context across boundaries.

## Section: Embeddings

An embedding is a numerical representation of text meaning. An embedding model converts a sentence, paragraph, or document chunk into a vector containing floating-point values. Texts with similar meanings tend to have nearby vectors.

## Section: PostgreSQL and pgvector

A RAG application can store document chunks and their embeddings in PostgreSQL. The pgvector extension adds vector data types and similarity-search capabilities. The application can compare a query embedding with stored document embeddings.

## Section: Semantic Search

Semantic search focuses on meaning rather than exact keyword matches. A question about finding relevant information can retrieve a chunk about vector similarity even when the exact words from the question are not present.

## Section: Clean Architecture

Clean Architecture separates business rules from infrastructure concerns. The domain layer remains independent of databases and external services. Application use cases define behavior, while infrastructure provides implementations.

## Section: CQRS

Command Query Responsibility Segregation separates state-changing commands from read-only queries. MediatR can dispatch commands and queries to their handlers.

## Section: Document Processing Pipeline

A typical RAG pipeline starts with a PDF or text upload, followed by metadata storage, text extraction, chunking, embedding generation, vector storage, and semantic retrieval.

## Section: Similarity

Vector similarity measures how close two embedding vectors are. Cosine similarity is commonly used for text embeddings. Higher similarity generally indicates that two pieces of text are more semantically related.

Test content 2: embeddings, vectors, chunks, semantic search, PostgreSQL, pgvector, retrieval, context, and language models are repeated intentionally so that chunking and semantic-search behavior can be tested.

# RAG Embedding Test — Page 3

## Section: Retrieval-Augmented Generation

Retrieval-Augmented Generation combines information retrieval with a large language model. The application first retrieves relevant information from documents and then provides that information to the language model as context.

## Section: Text Chunking

Long documents are divided into smaller chunks before embedding. Chunking makes semantic search more precise because each vector represents a manageable section of text. Overlapping chunks can preserve context across boundaries.

## Section: Embeddings

An embedding is a numerical representation of text meaning. An embedding model converts a sentence, paragraph, or document chunk into a vector containing floating-point values. Texts with similar meanings tend to have nearby vectors.

## Section: PostgreSQL and pgvector

A RAG application can store document chunks and their embeddings in PostgreSQL. The pgvector extension adds vector data types and similarity-search capabilities. The application can compare a query embedding with stored document embeddings.

## Section: Semantic Search

Semantic search focuses on meaning rather than exact keyword matches. A question about finding relevant information can retrieve a chunk about vector similarity even when the exact words from the question are not present.

## Section: Clean Architecture

Clean Architecture separates business rules from infrastructure concerns. The domain layer remains independent of databases and external services. Application use cases define behavior, while infrastructure provides implementations.

## Section: CQRS

Command Query Responsibility Segregation separates state-changing commands from read-only queries. MediatR can dispatch commands and queries to their handlers.

## Section: Document Processing Pipeline

A typical RAG pipeline starts with a PDF or text upload, followed by metadata storage, text extraction, chunking, embedding generation, vector storage, and semantic retrieval.

## Section: Similarity

Vector similarity measures how close two embedding vectors are. Cosine similarity is commonly used for text embeddings. Higher similarity generally indicates that two pieces of text are more semantically related.

Test content 3: embeddings, vectors, chunks, semantic search, PostgreSQL, pgvector, retrieval, context, and language models are repeated intentionally so that chunking and semantic-search behavior can be tested.

# RAG Embedding Test — Page 4

## Section: Retrieval-Augmented Generation

Retrieval-Augmented Generation combines information retrieval with a large language model. The application first retrieves relevant information from documents and then provides that information to the language model as context.

## Section: Text Chunking

Long documents are divided into smaller chunks before embedding. Chunking makes semantic search more precise because each vector represents a manageable section of text. Overlapping chunks can preserve context across boundaries.

## Section: Embeddings

An embedding is a numerical representation of text meaning. An embedding model converts a sentence, paragraph, or document chunk into a vector containing floating-point values. Texts with similar meanings tend to have nearby vectors.

## Section: PostgreSQL and pgvector

A RAG application can store document chunks and their embeddings in PostgreSQL. The pgvector extension adds vector data types and similarity-search capabilities. The application can compare a query embedding with stored document embeddings.

## Section: Semantic Search

Semantic search focuses on meaning rather than exact keyword matches. A question about finding relevant information can retrieve a chunk about vector similarity even when the exact words from the question are not present.

## Section: Clean Architecture

Clean Architecture separates business rules from infrastructure concerns. The domain layer remains independent of databases and external services. Application use cases define behavior, while infrastructure provides implementations.

## Section: CQRS

Command Query Responsibility Segregation separates state-changing commands from read-only queries. MediatR can dispatch commands and queries to their handlers.

## Section: Document Processing Pipeline

A typical RAG pipeline starts with a PDF or text upload, followed by metadata storage, text extraction, chunking, embedding generation, vector storage, and semantic retrieval.

## Section: Similarity

Vector similarity measures how close two embedding vectors are. Cosine similarity is commonly used for text embeddings. Higher similarity generally indicates that two pieces of text are more semantically related.

Test content 4: embeddings, vectors, chunks, semantic search, PostgreSQL, pgvector, retrieval, context, and language models are repeated intentionally so that chunking and semantic-search behavior can be tested.